

Supplementary Information for ldsc-rs: Exact and approximate LD Score Regression at biobank scale

Sharif Haason¹ and Yousef Khan²

¹Independent

²Independent

March 2026

This document collects supplementary tables referenced from the main text of ldsc-rs. Tables are numbered S1–S7 and each notes the main-text section it supports. The benchmark sweeps, complexity comparison, and per-subcommand feature inventory are placed here to keep the main-text table count focused on the headline results (parity, performance, the chunk-rounding finding, the cross-tool capability matrix, and the sketched- $\hat{h}^2$ unbiasedness simulation).

LD score regression | heritability | CountSketch | supplementary

✉ sharif@monark-markets.com

N	sketch-1000 + mask (h^2 truth cluster)	sketch-1000 + mask peak RSS (MB)	exact f32 wall (s)	exact f32 RSS (MB)	exact f64 wall (s)	exact f64 RSS (MB)
503	— ^a	—	9.4	634	15.5	714
10K	7.4 s	1,157	93 s	4,449	243 s	4,190
20K	6.4 s	1,444	133 s	7,630	482 s	5,716
50K	9.7 s	2,226	292 s	13,752	— ^b	—
100K	14.4 s	4,022	1{,}013 s	14,609	— ^b	—

Table S1: **Supports main-text §“Biobank-scale performance” (scaling figure).** Time and memory scaling across N on a single Apple M5 Pro laptop (M5 Pro, 18 cores, 36 GB RAM, macOS 15). Wall time and peak RSS for the **h^2 truth-cluster sketch mode** (`--sketch 1000 --snp-level-masking`), which recovers per-SNP exact heritability estimates within 0.001 across BMI and Height GWAS, against the two exact modes. At $N = 100\{, \}$ 000 the truth-cluster sketch delivers a $\sim 70\times$ speedup over exact f32 (14.4 s vs. 1,013 s) and a $\sim 3.6\times$ memory advantage (4.0 GB vs. 14.6 GB). At smaller N the truth-cluster sketch is essentially indistinguishable in wall time from cheaper but biased sketch dimensions (e.g. `--sketch 200` finishes in 13 s at $N = 100,000$ but gives h^2 off the truth cluster by $\{\sim\}$ 0.013 on real GWAS), so there is no speed reason to prefer a low d over the recommended truth-cluster $d = 1000$. ^(a) `--sketch 1000` requires $d \leq N$ and is not applicable at $N = 503$ — at 1000 Genomes scale, exact f32 is the recommended mode. ^(b) Exact f64 at $N \geq 50,000$ takes multi-hour walls locally; the AWS EPYC 7R13 figure of 727 s at $N = 50,000$ is reported in the main-text biobank performance table.

d	Measured r	1000G time (s)	Biobank time (s)	Biobank speedup
50	0.842	3.6 [†]	16	41×
200	0.960	5.0 [†]	16	41×
500	0.981	7.9 [†]	18	36×
1000	0.990	10.5 [†]	20	33×
2000	0.994	28.3 [†]	22	30×
5000	0.996	—	36	18×
10000	0.997	—	59	11×

Table S2: **Supports main-text §“CountSketch accuracy–performance tradeoff”**. CountSketch accuracy and runtime vs. sketch dimension. Accuracy (r): Pearson correlation with exact f64 LD scores, measured on the synthetic biobank dataset ($N = 50\{, \}000$, 1.66M SNPs; see main-text §Limitations). Times from the cost model $T(d) = T_{\text{scatter}} + T_{\text{GEMM}} \cdot d$ except where directly measured. [†]1000G local benchmarks (Ryzen 5 5600X). Biobank speedup vs. exact f64 (727 s).

N	Recommended d	Wall (s)	Pearson r	$ \hat{h}_{\text{sketch}}^2 - \hat{h}_{\text{exact}}^2 $
503	500	3.0	0.994	0.006
10K	1\{, \}000	7.4	0.994	0.003
20K	1\{, \}000	6.4	0.993	0.003
50K	1\{, \}000	9.7	0.994	0.000
100K	1\{, \}000	14.4	0.993	0.002

Table S3: **Supports main-text §“Optimal d as a function of N ”**. Sweet-spot d per N (criterion: smallest d giving Pearson $r \geq 0.993$ and $\hat{h}^2$ within one regression-SE of exact), measured with `--snp-level-masking` on Apple M5 Pro. The optimum is approximately N -independent at $d \approx 1\{, \}000$ because the sketch error is set by intra-window LD structure (window size ~ 800 -1500 SNPs), not by sample size. Wall scales mildly with N from the fused $O(Nc)$ scatter cost; at $N = 100\{, \}000$ the recommended setting still finishes in ~ 14 s, vs. ~ 17 min for exact f32 ($\sim 70 \times$ speedup). For applications where exact per-SNP LD scores are critical (partitioned heritability, fine-mapping), bump to $d = 5\{, \}000$ for $r \geq 0.999$ at the cost of ~ 30 s wall.

<code>--chunk-size</code>	mean L2	Δ L2 vs c=50	$\hat{h}^2$ at $h^2=0.2$	$\hat{h}^2$ at $h^2=0.5$
25	18.71	−0.13%	0.2115 (+5.8%)	0.5580 (+11.6%)
50 (Python default)	18.73	—	0.2115 (+5.7%)	0.5569 (+11.4%)
100	18.77	+0.20%	0.2109 (+5.4%)	0.5556 (+11.1%)
200 (ldsc-rs default)	18.85	+0.62%	0.2107 (+5.3%)	0.5528 (+10.6%)
500	18.91	+0.95%	0.2097 (+4.8%)	0.5532 (+10.6%)
1000	19.05	+1.68%	0.2092 (+4.6%)	0.5526 (+10.5%)

Table S4: **Supports main-text §“The `--chunk-size` knob and implementation parity”**. LD-score and heritability sensitivity to `--chunk-size`, computed on chromosome 22 of 1000 Genomes EUR ($N = 503$, 18,627 SNPs after $\text{MAF} \geq 0.05$, `--ld-window-kb 1000`, `--global-pass`). Mean LD score grows monotonically with chunk size (range 1.8% of mean) due to chunked over-counting; $\hat{h}^2$ correspondingly decreases. The full range of the chunk-size effect on $\hat{h}^2$ is ~ 1.1 percentage points of relative bias—small but systematic.

Aspect	GCTA	Python LDSC	ldsc-rs
Window eviction	Per-SNP exact	c-SNP chunks, c=50	c-SNP chunks, c=200 (default); per-SNP exact with <code>--snp-level-masking</code>
Block boundaries	Two-pass overlap averaging	Single forward pass	Single forward pass
r ² estimator default	Biased (+noise floor)	Unbiased	Unbiased
Unbiased r ² flag	<code>--ld-score-adj</code>	n/a (always unbiased)	n/a (always unbiased)
Parallelism	OpenMP per SNP	Serial	Rayon per chromosome + SIMD GEMM via faer
Python-LDSC parity flag	—	—	<code>--python-compat</code>

Table S5: **Supports main-text §“Three implementations: GCTA, Python LDSC, ldsc-rs”**. Algorithmic choices across the three independent LD-score implementations. All three share the same theoretical definition $\ell_j = \sum_k r_{jk}^2$ but differ in windowing, edge handling, and r² estimator. GCTA’s per-SNP exact semantics match the LDSC paper’s mathematical statement; Python LDSC’s chunked approximation is undocumented in the paper but inherited by every Python fork. ldsc-rs’s `--snp-level-masking` reproduces GCTA’s per-SNP semantics; `--python-compat` produces bit-identical Python LDSC output (`max_abs_diff = 0` verified on chr22 1000G EUR).

Subcommand	Status	Notes
<code>munge-sumstats</code>	✓	Polars streaming; <code>--daner/--daner-n</code> ; INFO-score handling (<code>--info/--info-min</code>) identical to Python LDSC
<code>l2</code> (LD scores)	✓	<code>--sketch</code> , <code>--fast-f32</code> , <code>--snp-level-masking</code>
<code>h2</code> (heritability)	✓	<code>--overlap-annot</code> , <code>--h2-cts</code> , <code>two-step</code>
<code>rg</code> (genetic corr.)	✓	<code>--intercept-h2</code> , <code>multi-trait</code>
<code>make-annot</code>	✓	BED interval annotation
<code>cts-annot</code>	✓	Continuous annotation binning

Table S6: **Supports main-text §“Feature parity”**. Feature parity between ldsc-rs and Python LDSC. All subcommands and major flags are supported. ldsc-rs adds `--sketch`, `--fast-f32`, and `--snp-level-masking` as new modes not present in the original.

Supplementary Tables

Method	Sketch cost	GEMM cost
Exact	—	$O(Ncw)$ per chunk
Gaussian sketch	$O(dNc)$	$O(dcw)$
CountSketch	$O(Nc)$	$O(dcw)$

Table S7: **Supports main-text Methods §“Complexity comparison”**. Asymptotic cost comparison per chunk. N : individuals, c : chunk size, w : window size, d : sketch dimension. CountSketch achieves input sparsity time for the projection step. For PLINK BED genotypes (dense; $\text{nnz} \approx Nc$), the distinction between $O(\text{nnz})$ and $O(Nc)$ vanishes, but the constant-factor advantage of scatter-add (one memory write per entry) over dense matrix–vector products (d multiply-accumulates per entry) remains critical. Gaussian sketch is shown for reference; ldsc-rs implements CountSketch only.